NESTJS CHEAT SHEET

Core setup, initialization, operations & integration quick reference

PART 1: SETUP & CORE

01. SETUP & CLI

Installation

```
npm i -g @nestjs/cli
nest new my-app
nest generate module users
nest generate controller users
nest generate service users
npm run start:dev  (watch mode)
```

02. MODULES

Architecture

```
@Module({
imports: [TypeOrmModule.forFeature([User])],
controllers: [UsersController],
providers: [UsersService],
exports: [UsersService],
})
export class UsersModule {}
```

Feature modules keep concerns isolated

03. CONTROLLERS

YAML / Config

```
@Controller('users')
export class UsersController {
constructor(private readonly svc: UsersServic...
@Get() findAll(): Promise<User[]> { return th...
@Get(':id') findOne(@Param('id') id: string) ...
@Post() @HttpCode(201) create(@Body() dto: Cr...
@Put(':id') update(@Param('id') id, @Body() d...
@Delete(':id') remove(@Param('id') id) { ... }
```

04. PROVIDERS & SERVICES

Workflow

```
@Injectable()
export class UsersService {
constructor(
@InjectRepository(User)
private readonly repo: Repository<User>
) {}
async findAll(): Promise<User[]> {
return this.repo.find()
}
}
```

05. DTOs & VALIDATION

CRUD API

```
import { IsEmail, IsString, MinLength } from ...
export class CreateUserDto {
@IsString() @MinLength(2)
name: string
@IsEmail()
email: string
}
// main.ts:
app.useGlobalPipes(new ValidationPipe({ white...
```

06. GUARDS

Integration

```
@Injectable()
export class JwtAuthGuard extends AuthGuard('...
@UseGuards(JwtAuthGuard)
@Get('profile')
getProfile(@Request() req) { return req.user }
@Injectable()
export class RolesGuard implements CanActivat...
canActivate(ctx: ExecutionContext): boolean {...
}
```

SECURITY, MONITORING & OPERATIONS

Production hardening, observability, performance tuning & CLI reference

PART 2: OPS & SECURITY

07. INTERCEPTORS

Hardening

```
@Injectable()
export class LoggingInterceptor implements Ne...
intercept(ctx: ExecutionContext, next: CallHa...
console.log('Before...')
return next.handle().pipe(
tap(() => console.log('After...'))
)
}
}
@UseInterceptors(LoggingInterceptor)
```

08. PIPES

Observability

```
@Get(':id')
findOne(@Param('id', ParseIntPipe) id: number...
// Custom pipe:
@Injectable()
export class ParseIdPipe implements PipeTrans...
transform(val: any): number {
const n = parseInt(val)
if (isNaN(n)) throw new BadRequestException()
return n
}
}
```

09. MIDDLEWARE

Optimization

```
@Injectable()
export class LoggerMiddleware implements Nest...
use(req: Request, res: Response, next: Functi...
console.log(`${req.method} ${req.url}`)
next()
}
}
// In module:
configure(consumer: MiddlewareConsumer) {
consumer.apply(LoggerMiddleware).forRoutes('*...
}
```

10. EXCEPTION FILTERS

Unit Tests

```
@Catch(HttpException)
export class HttpExceptionFilter implements E...
catch(ex: HttpException, host: ArgumentsHost)...
const ctx = host.switchToHttp()
const res = ctx.getResponse<Response>()
res.status(ex.getStatus()).json({
statusCode: ex.getStatus(),
message: ex.message
})
}
}
```

11. COMMON DECORATORS

Commands

```
@Controller @Get @Post @Put @Delete @Patch
@Param @Body @Query @Headers @Request @Respon...
@Injectable @Module @Guard @UseGuards @UseInt...
@Catch @UseFilters  @UsePipes
@SetMetadata('roles', ['admin'])
@Reflector + Reflector.getAllAndOverride()
```

12. COMMON GOTCHAS

Warning

Circular dependency: use forwardRef(() => ServiceB)
ValidationPipe whitelist:true strips unknown props
Global guards/pipes must be registered in main.ts
TypeORM relations must be explicitly loaded or use eager
Async providers: use async factory or useFactory